MODULE 1 · SW DESIGN

Introduction to Software Design & Architecture

Two words we use every day — and rarely define. Let's fix that.

Akın Kaldıroğlu · Lead Trainer

COURSE MAP

Where We Are

This is the first stop in Phase 1 — Software Design. Everything we build in the next eight weeks stands on the ideas in this session.

You are here

Module 1 — the vocabulary and mindset of design & architecture.

Next

Fundamentals of SW Design — cohesion, coupling & change.

The goal

Learn to make — and defend — structural decisions, with AI as your partner.

WHAT YOU'LL LEAVE WITH

Learning Objectives

- Explain what software design is — and why it is decision-making, not typing.
- Explain what software architecture is — and how it differs from design.
- See design and architecture as one continuum, governed by the same forces.
- Understand why design skill matters more, not less, in the age of AI.

SECTION 01

Why Design at All?

Nobody designs software for fun. We design because software changes.

THE NATURE OF THE BEAST

Software Is Soft

The name is a promise: software is meant to be changed. The hard part was never writing it the first time — it is changing it safely, again and again.

Requirements move

Markets, users and regulations never sit still.

Teams change

The person who reads the code is rarely the one who wrote it.

Time compounds

Most of a system's life — and cost — comes after version 1.0.

FOLLOW THE MONEY

Where the Cost Really Is

Studies and hard experience agree: the majority of total software cost is spent after the first release — reading, understanding, and changing code.

Writing code once

- Fast, exciting, visible
- A small slice of the timeline
- What juniors optimise for

Living with code

- Reading it to understand it
- Changing it without breaking things
- Where design pays — or punishes — you

📖 **Software Engineering at Google** — *"programming integrated over time" — a programmer writes code; an engineer owns it as it changes. The difference is time.*

THE ALTERNATIVE

What Happens Without Design

Skip design and you don't get 'no architecture' — you get an accidental one. It usually has a name:

Rigidity

Every change forces a cascade of other changes.

Fragility

A change here breaks something unrelated over there.

The Big Ball of Mud

No discernible structure — everything knows everything.

📖 **A Philosophy of Software Design (Ousterhout)** — *complexity has two sources — coupling and ambiguity; ambiguity breeds cognitive load and incomprehensible structure.*

WHAT GOOD AIMS FOR

Software Quality Goals

Design and architecture are how we pursue quality attributes. This program focuses on two of them above the rest:

Our focus

- Maintainability / changeability — change safely & cheaply
- Scalability — grow with load and with the team

Also on the radar

- Security · Performance
- Availability · Testability
- Usability · Portability

SECTION 02

What Is Software Design?

Design is the set of decisions that give code its shape.

A WORKING DEFINITION

Design Is Decision-Making

Software design is the activity of making structural and behavioural decisions — how to divide the problem into parts, give each part a clear job, and connect the parts — so the system meets its requirements and stays easy to change.

"The code is the result of design. The design is the **thinking** that produced it."

THE SMALL SCALE

Functional (Low-Level) Design

At the level of code, design answers a handful of recurring questions:

- What are the parts? — functions, classes, modules.
- What is each part responsible for? — one clear job, ideally.
- How do the parts depend on one another? — who calls whom, who knows what.
- How will this change next month? — and does the shape make that easy?

EVERY SCALE

Design Happens at Every Level

From a single expression to a whole subsystem, the same act repeats: name things, assign responsibility, manage dependencies.

A line

Naming a variable well is a design decision.

A function

Choosing what it does — and does not — do.

A class

Grouping data and behaviour that belong together.

A module

Drawing a boundary others depend on.

MAKE IT CONCRETE

▶ runnable `dev.kaldiroglu.bootcamp.intro`

Same Feature, Two Designs

Compute an order's total. Version A mixes everything into one place; Version B separates the decision (discount policy) from the calculation.

Java — *Version A — everything tangled together*

```
// A — one method knows tax, discounts, shipping, formatting...
double total(Order o) {
    double t = 0;
    for (Item i : o.items()) t += i.price() * i.qty();
    if (o.customer().isVip()) t *= 0.9;        // discount rule
    t += t * 0.20;                             // tax rule
    return t;                                  // hard to test, hard to change
}
```

MAKE IT CONCRETE

▶ runnable `dev.kaldiroglu.bootcamp.intro`

Separate the Decision from the Work

Version B — pull the discount rule out behind a small boundary. Same idea in three languages:

Java

```
interface Discount {
  double apply(double t,
               Customer c);
}

double total(Order o,
     Discount d) {
  double t = subtotal(o);
  return withTax(
     d.apply(t, o.customer()));
}
```

C#

```
interface IDiscount {
  double Apply(double t,
               Customer c);
}

double Total(Order o,
     IDiscount d) {
  var t = Subtotal(o);
  return WithTax(
     d.Apply(t, o.Customer));
}
```

Python

```
class Discount(Protocol):
  def apply(self, t, c):
      ...


def total(o, d):
  t = subtotal(o)
  return with_tax(
     d.apply(t, o.customer))
```

Now the discount rule can change — VIP, seasonal, none — without touching the calculation. That is design.

WHAT WE'RE AIMING FOR

What 'Good' Design Feels Like

We'll make these precise next session. For now, the intuition:

Understandable

You can read a part and know what it does.

Changeable

A new requirement touches few, obvious places.

Testable

You can check a part in isolation.

Reusable

A part is useful beyond where it was born.

SECTION 03

What Is Software Architecture?

Architecture is the set of decisions that are expensive to reverse.

A WORKING DEFINITION

Architecture Is the Big Decisions

Software architecture is the set of significant decisions about a system — its major parts, how they relate, and the principles guiding its design and evolution. 'Significant' means: costly to change later.

> “Architecture is the decisions you wish you could get right early, because they are painful to change late.”

THE BUILDING BLOCKS

Components, Connectors, Constraints

Components

The major parts — services, layers, subsystems that hold responsibility.

Connectors

How parts talk — calls, queues, APIs, shared data.

Constraints

The rules everyone follows — 'the UI never touches the database directly'.

WHY IT EXISTS

Architecture Serves System Qualities

Design mostly shapes how easy the code is to work with. Architecture mostly decides whether the system can meet its big, cross-cutting promises:

Runtime qualities

- Performance & scalability
- Availability & reliability
- Security

Structural qualities

- Maintainability & evolvability
- Deployability & testability
- Team autonomy

WHAT THEY LOOK LIKE

Examples of Architectural Decisions

- Monolith or distributed system? (We'll weigh this in Phase 2.)
- How do we layer the system — and what may depend on what?
- Synchronous calls or asynchronous messages between parts?
- One database or many? SQL, NoSQL, or both?
- Which cross-cutting rules are non-negotiable — security, logging, boundaries?

Note: we only name these today — each gets its own session later.

SECTION 04

Design vs Architecture

Not two different things — the same thing at two altitudes.

THE HEART OF IT

It's About Scope & Cost of Change

> “How expensive is it to change my mind later?”

Cheap to reverse → Design

- Rename a method, split a class
- Swap an algorithm behind an interface
- Local, contained, low blast-radius

Costly to reverse → Architecture

- Split a monolith into services
- Change the database paradigm
- System-wide, high blast-radius

A USEFUL PICTURE

A Continuum, Not a Wall

There is no hard line where design ends and architecture begins. It is a spectrum of significance — and the same forces run through all of it.

The same two forces — **cohesion** and **coupling** — decide quality at every point on this line.

SIDE BY SIDE

Design vs Architecture

	Software Design	Software Architecture
Scope	Classes, functions, modules	Whole system & its major parts
Concern	How code is structured	How the system meets its qualities
Decisions	Detailed, local	Significant, structural
Cost to change	Usually low	Usually high
Horizon	Days to weeks	Months to years
Example	Extract an interface	Choose microservices

AN ANALOGY (HANDLE WITH CARE)

Building a House

Architecture

- Where the load-bearing walls go
- How many floors; where the stairs are
- Moving these later = knock down walls

Design

- Cabinet layout, paint, fixtures
- Comfortable, but not structural
- Change these on a weekend

Analogies leak — software is softer than concrete — but the intuition about 'cost to change' holds.

WHY YOU NEED BOTH

Neither One Saves You Alone

Great architecture, bad design

The right boxes, full of unreadable, untestable code. It rots from the inside.

Great design, bad architecture

Beautiful classes trapped in a system that can't scale, deploy, or evolve.

Both, together

Parts that are a pleasure to change, inside a system that meets its promises.

HOW THIS COURSE WORKS

Functional First, Then Architecture

A question we hear a lot: when do we build the big project? Our path — master design where it's most tangible (the functional level), then scale the very same criteria up to architecture.

Start here — functional design

- One method, one class at a time
- Core criteria: cohesion & coupling
- Clean code, SOLID, patterns, tests

Then — architectural design

- Components, layers, services, boundaries
- The same criteria, at system scale
- Layered, Hexagonal / Onion / Clean

Design is fractal: cohesion and coupling read the same at one class or a hundred. Learn them small, and they carry you to the large — the project arrives once the fundamentals are yours.

SECTION 05

Design in the Age of AI

This is an agentic bootcamp. So where do you fit?

THE SHIFT

AI Writes Code. You Make Decisions.

AI can produce a plausible implementation in seconds. What it cannot do for you is decide whether that implementation is the right shape for your system.

AI is great at

- Generating code fast
- Boilerplate & translations across languages
- Recalling APIs and patterns

You own

- Framing the problem & constraints
- Judging structure: cohesion, coupling, boundaries
- The costly, architectural calls

THE PUNCHLINE

Design Skill Matters More, Not Less

When anyone can generate code, the scarce skill is judging it. To review, steer and correct an AI, you need the very vocabulary this program teaches.

- You can only accept or reject a design if you can recognise a good one.
- You are the decision-maker; the AI is a fast, tireless implementer.
- Prompts encode intent — design thinking is what makes intent precise.
- We'll practise this directly in 'Agentic Design & Coding' with Claude Code.

THE FIVE THINGS TO REMEMBER

Key Takeaways

- ✓ We design because software changes — most cost comes after v1.
- ✓ Design is decision-making: parts, responsibilities, dependencies.
- ✓ Architecture is the subset of decisions that are costly to reverse.
- ✓ Design and architecture are one continuum — cohesion & coupling run through both.
- ✓ With AI, design judgement is the differentiator. You decide; it implements.

GO DEEPER

Further Reading

The books behind this session. If you read only one, read the first.

📖 **The Pragmatic Programmer, 20th Anniv. ed.** Hunt & Thomas · Addison-Wesley, 2019

The habits behind everything that follows — orthogonality, DRY, tracer bullets.

📖 **Fundamentals of Software Architecture, 2nd ed.** Richards & Ford · O'Reilly, 2025

What architecture is, how it differs from design, and why every decision is a trade-off.

📖 **The Mythical Man-Month, Anniv. ed.** Frederick P. Brooks Jr. · Addison-Wesley, 1995

"No Silver Bullet" — essential vs. accidental complexity, the idea this course rests on.

NEXT SESSION

What's Next

MODULE 2 · TUESDAY 14 JULY

Fundamentals of Software Design

with Akın Kaldıroğlu

Cohesion, coupling & change — and the Law of Demeter. The tools to say precisely why one design is better than another.

FURTHER PRACTICE

The Codebase You'll Refactor

These ideas come alive in the ACME Orders project — the smelly codebase behind every exercise, in all three languages.

Refactoring — A Principle-Driven Course

Repo `github.com/kaldiroglu/refactoring-course-student-materials`

IN THIS SESSION, LOOK AT

- `Projects/RefactoringCapstone-Java, -CSharp, -Python`
- `Slides-PDF/Ch01-Introduction.pdf`

Questions?

Let's design software worth maintaining.